# JTS Topology Suite

## TestRunner & TestBuilder User Guide

Amended for the curve stack on this fork

Base text: 3 February 2005 · Amendment: 16 August 2026

Against feature/sfa-curve-rgr @ 8e39e39a · PR #7 is the source of truth

*This is a JTS guide. The 2003–2005 Vivid Solutions text is kept where it is still true. Sections that would lie about curves, I/O, overlay, Hausdorff, or TestBuilder on this fork are amended against* `feature/sfa-curve-rgr` *at* `8e39e39a` *(PR #7).*

*LocationTech JTS remains the upstream project. This fork is not described here as if it were already upstream. Package names in this tree are* `org.locationtech.jts`*; the 2003–2005 copies used* `com.vividsolutions.jts`*.*

## Document change control

| Revision | Date | Author | Change |
|---|---|---|---|
| **1.0** | 13 Sep 2001 | Jonathan Aquino | Original draft. |
| **1.1** | 17 Dec 2001 | Jonathan Aquino | Added TestBuilder documentation. |
| **1.2** | 30 Jan 2002 | Jonathan Aquino | Added ACME licence (historical). |
| **1.2-fork** | 16 Aug 2026 | This fork | Amend startup, packages, curve drawing, WKT/WKB, OverlayNGCurve, Hausdorff. Replace ACME appendix with the actual JTS dual licence. Do not claim unmerged TestBuilder drafts as shipped. |

**Honesty lock (this amendment)**

• Verified against the #7 tree at 8e39e39a. Unmerged drafts are not described as shipped. This document does not restack onto other bars.

• TestBuilder exists on this tree and can draw CircularString, CompoundCurve, and CurvePolygon (toolbar tools plus WKT via CurveWKTReader). It also exposes OverlayNGCurve and the distance functions. This guide does not claim logo-as-curves, A/B draw-tool icon honesty, or a laser compound hull as shipped on #7 — those are not on this tree.

• License remains the JTS dual licence: Eclipse Public License 2.0 and Eclipse Distribution License 1.0. See LICENSE_EPLv2.txt, LICENSE_EDLv1.txt, and LICENSES.md. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

## Table of contents



## 1. Overview

This guide describes how to use the JTS TestRunner, a Java application that validates the JTS Topology Suite, and the JTS TestBuilder, a Java application that creates, visualizes, and runs geometry tests.

The TestRunner and TestBuilder are used by people who are developing or evaluating JTS. While similarly named, the JTS TestRunner is not the JUnit TestRunner. JUnit is a general Java testing framework. The JTS TestRunner is easier for testing JTS computations because tests are specified in XML text files — no Java coding or compilation. Those files can be created by hand or generated by the TestBuilder.

The 2005 guide required Java 2 JRE/SDK v1.3. This tree targets Java 8 and above. Download a current JDK from the vendor of your choice; the 2005 java.sun.com/products link is historical.

The 2005 class names were `com.vividsolutions.jtstest.testrunner.TopologyTestApp` and `com.vividsolutions.jtstest.testbuilder.JTSTest`. On this tree they are `org.locationtech.jtstest.testrunner.JTSTestRunnerCmd` and `org.locationtech.jtstest.testbuilder.JTSTestBuilder`. The usual way to start TestBuilder is the built jar, not a hand-built classpath of jdom.jar / xerces.jar / jts.jar / jts_test.jar.

## 2. Using the TestRunner

The TestRunner runs XML test files that validate JTS. Test files can be created with the TestBuilder (§3) or by hand (§5).

### 2.1 Starting

From a built tree, run the TestRunner via the scripts in `bin/` or:

```
java -cp modules/tests/target/jts-tests-*-SNAPSHOT.jar
     org.locationtech.jtstest.testrunner.JTSTestRunnerCmd
     -files path/to/tests.xml
```

The 2005 GUI mode (`-gui` / testrunner.bat) and the switches `-files`, `-properties`, `-verbose` are the same idea: a list of XML files, a results panel or text listing, and a status line of cases / tests / passed / failed / exceptions.

### 2.2 Reading the results

A **case** specifies two geometries (A and B). A **test** specifies an operation and its expected value. The status line reports counts of cases, tests, passes, failures, thrown exceptions, and parse exceptions.

| Statistic | Meaning |
|---|---|
| **Cases** | Each case specifies two geometries to test. |
| **Tests** | Each test specifies an operation on those geometries. |
| **Passed** | Actual value matches expected value. |
| **Failed** | Actual value differs from expected value. |
| **Exception** | An error occurred inside JTS or the TestRunner. |
| **Parse exception** | The test file has a syntax error. |

Passed-test detail is shown only with `-verbose`. A failed test means either fix the test or fix JTS. A parse exception is a problem in the XML or WKT (a missing comma in a LINESTRING is the 2005 example and is still the usual cause).

### 2.3 Switches

| Switch | Description |
|---|---|
| **-files** | List of test files and directories. Separate items with spaces. |
| **-properties** | A .properties file that remembers the last file list. |
| **-gui** | Graphical mode, when the build still ships that entry point. |
| **-verbose** | Show passed tests as well as failures. |

The easiest way to specify every file in a directory is to pass the directory to `-files`.

> **Amendment — TestRunner and curves**
>
> • XML <a> / <b> bodies that contain CIRCULARSTRING / COMPOUNDCURVE / CURVEPOLYGON / MULTICURVE / MULTISURFACE must be parsed with a curve-capable reader. On this tree the TestRunner / TestCase path uses CurveWKTReader.
>
> • A test that calls Geometry.intersection on a curve is not OverlayNGCurve. Name the OverlayNGCurve function if that is what you mean to assert.
>
> • A DiscreteHausdorffDistance assertion is the public API: two certified pairs are closed-form; everything else is chords. fail() stays.

# 3. Using the TestBuilder

The 2005 guide called TestBuilder experimental, with a useful starting point for creating, visualizing, and running test files. That description is still fair. The application on this tree is the current LocationTech JTS TestBuilder plus the curve tools that are actually merged on #7.

## 3.1 Starting the TestBuilder

Build the app module, then from the project root:

```
java -jar modules/app/target/JTSTestBuilder.jar -Xmx2000M
```

On macOS, if the native look-and-feel misbehaves:

```
java -Dswing.defaultlaf=javax.swing.plaf.metal.MetalLookAndFeel \
     -jar modules/app/target/JTSTestBuilder.jar -Xmx2000M
```

Do not start it as the 2005 `java com.vividsolutions.jtstest.testbuilder.JTSTest` with a four-jar classpath. That layout is gone.

## 3.2 Opening a test file

File / Open XML File(s). The TestBuilder displays one test case at a time: a pair of geometries and their tests. Use Previous / Next or the Tests list to move. Pan, zoom, zoom 1:1, and zoom-to-extent examine the geometries. The WKT tab shows Well-Known Text.

## 3.3 Precision model

A test file's precision model is set from the TestBuilder precision-model dialog. The default is Floating. See the Technical Specifications.

## 3.4 Creating geometries

The 2005 workflow is still the right picture: pick geometry A or B, pick a draw tool, click in the view. Geometry A is shown in one colour, geometry B in the other. You can also type WKT. Erase, undo point, and undo part still apply.

On this tree the draw tools include the linear set (point, linestring, polygon, rectangle) and the curve set that is actually merged: CircularString, CompoundCurve, CurvePolygon. Triangle and TIN tools also exist. WKT apply uses CurveWKTReader, so CIRCULARSTRING pasted into the WKT tab is a CircularString, not a parse error.

## 3.5 Predicates and functions

The 2005 Predicates tab (expected DE-9IM, binary predicates) and Functions tab (union, intersection, buffer, …) are the same idea. The current UI is a function tree: pick a function, run it, inspect the result geometry and its WKT. OverlayNGCurve is registered as its own function class (intersection / union / difference / symDifference). That is the curve overlay surface. Geometry.intersection in the Geometry function group is not OverlayNGCurve.

## 3.6 Saving

File / Save As XML writes the current cases. The 2005 "Save As HTML" bulk documentation generator is not something this amendment claims as a supported workflow. The vividsolutions.com/jts/tests URL in the 2005 copy is historical.

# 4. Curves in TestBuilder (amendment)

TestBuilder exists on this tree and can draw CircularString, CompoundCurve, and CurvePolygon (toolbar tools plus WKT via CurveWKTReader). It also exposes OverlayNGCurve and the distance functions. This guide does not claim logo-as-curves, A/B draw-tool icon honesty, or a laser compound hull as shipped on #7 — those are not on this tree.

On this fork the concrete SFA/SQL-MM curve types live in the `jts-curve` module (`org.locationtech.jts.geom.curve`): **CircularString**, **CompoundCurve**, **CurvePolygon**, and the

collections **MultiCurve** and **MultiSurface**. Construction is opt-in: the default `GeometryFactory` throws on `createCircularString` / `createCompoundCurve` / `createCurvePolygon` / `createMultiCurve` / `createMultiSurface`. Use `CurveGeometryFactory`. Constructors do not linearise; there is no Linearizable-on-construct.

Core `WKTReader` still rejects `CIRCULARSTRING` and the other SQL-MM keywords. Read them with `CurveWKTReader`. Write structured members with `CurveWKTWriter`. Core `WKTWriter` already emits the subclass keyword via `getGeometryType()` for a flat CircularString; composite types need the curve writer to keep member tags.

WKB type codes 8–12 are first-class in core `WKBReader` and `WKBConstants` (CircularString=8, CompoundCurve=9, CurvePolygon=10, MultiCurve=11, MultiSurface=12). Construction still requires a curve-capable factory; otherwise the reader throws. `CurveWKBWriter` emits codes 8–12. A bare `WKBWriter` still walks the parent `instanceof` ladder and writes a CircularString as LineString (code 2) and a CurvePolygon as Polygon (code 3). That flatten-on-direct-write behaviour is what this tree does. Default/export-path honesty is not claimed here.

The curve overlay type is **OverlayNGCurve** — never OverlayNGCurved. It lives in `jts-curve` (`org.locationtech.jts.operation.overlayng.curve`). There is no public noder. Closed-form answers are limited to certified discs and named kits. Anything else densifies and delegates to core OverlayNG. Geometry.`intersection` / `union` / `difference` / `symDifference` on a curve type are not a general circular overlay.

Public `DiscreteHausdorffDistance` on this tree has a closed form for **two pairs only**: (1) a single-arc CircularString toward a single-segment LineString, apex $\sqrt{949}/6 - 7/6 \approx 3.967641$ for the witness `CIRCULARSTRING (0 0, 2 3, 10 0)` vs `LINESTRING (0 0, 10 0)`; (2) two circular discs (full-circle CurvePolygon). A MultiSurface unwrap of one disc is the same disc pair. The exact path skips densify; densify is not the laser. For every other pair the public API still sees vertices / chords. Tests keep `fail()`. This document does not claim DirectedHausdorffDistance replace, port, or ship. Fréchet remains open.

Closed-form lasers on this tree are certified discs and named kits only. Do not call buffer of the JTS logo, a compound hull, or an open-arc buffer a closed-form laser. Disc buffer of a full circular disc is a disc of radius r+d (or empty if r+d ≤ 0). An open arc stays on BufferOp chords. Clothoid hull, when a clothoid member is present, is a linear fallback — not a fail, and not a laser.

What this section does **not** claim, because it is not on #7: the JTS logo helper rewritten as first-class curves; A/B draw-tool icon honesty as a finished change; a laser convex hull of circular-plus-straight members; a Bar-2 DCEL / general circular noder. If a later draft lands those, amend this PDF again against that tree. Do not read them into #7.

## 5. Appendix: test file format

A test file is XML. It contains one or more cases; each case contains one or more tests. A case specifies two geometries. A test specifies an operation and its expected result.

### 5.1 Example

```
<run>
  <desc>example</desc>
  <precisionModel type="FLOATING"/>
  <case>
    <desc>point in a polygon</desc>
    <a>POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))</a>
    <b>POINT (5 5)</b>
    <test>
      <op name="contains">true</op>
    </test>
  </case>
  <case>
    <desc>single-arc D-HF witness</desc>
    <a>CIRCULARSTRING (0 0, 2 3, 10 0)</a>
```

```
    <b>LINESTRING (0 0, 10 0)</b>
    <test>
      <op name="orientedDistance" arg1="a" arg2="b">3.967641</op>
    </test>
  </case>
</run>
```

The second case is the D-HF closed-form witness on this tree (apex $\sqrt{949}/6 - 7/6$). A case that is not one of the two certified pairs must not be written as if DiscreteHausdorffDistance were arc-length Hausdorff. The first case is the 2005 example, unchanged.

### 5.2 XML shape

The 2005 DTD-style listing is still the right shape: `<run>` contains `<precisionModel>` and one or more `<case>`; each case has `<a>`, optional `<b>`, and `<test>` / `<op>` with a name and expected value. WKT inside `<a>` / `<b>` may now be a curve keyword; the reader on this tree is CurveWKTReader.

## 6. Appendix: licence

License remains the JTS dual licence: Eclipse Public License 2.0 and Eclipse Distribution License 1.0. See LICENSE_EPLv2.txt, LICENSE_EDLv1.txt, and LICENSES.md. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

The 2005 "Appendix: ACME Licence" is not reproduced. It was never the JTS project licence on this tree.

> *End of TestRunner & TestBuilder User Guide amendment. Screen-shot figures from the 2005 Amyuni conversion are omitted; the procedures they illustrated are kept in prose.*